Reporte

Nombre:	**Matrícula**:
Landon Alexander Bautista Hernandez	2101662
Karla Judith Baez Villanueva	2107930
Alexa Daiana Amaya Torres	2100425

Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 24	**Actividad**: ACT-F03. Diseño Arquitectural del Sistema
Fecha: 16/10/2026	
Bibliografía (Formato APA) Archivo “03_Sazón_Urbano.docx” proporcionado en la plataforma TEAMS	

Objetivo:

Diseñar una estructura de software para Sazón Urbano a partir de los requisitos y procesos identificados en la Unidad 2, definiendo cómo se organizarán sus componentes y en qué parte del sistema se gestionarán las reglas y funciones del negocio antes de iniciar su desarrollo.

Procedimiento:

Diseño del sistema

Caso: sazón urbano

Este trabajo retoma el análisis realizado en la Unidad 2 sobre Sazón Urbano, un restaurante que ofrece atención en mesa, pedidos para llevar y entregas a domicilio. Durante esa unidad se identificaron las necesidades del negocio mediante un documento de requerimientos (BRD), 20 historias de usuario con sus respectivos criterios de aceptación, un glosario de términos y la representación de sus procesos mediante BPMN. Ahora, el objetivo es establecer cómo se organizará el software, dónde se aplicarán las reglas del negocio y de qué manera interactuarán sus diferentes componentes antes de comenzar la programación.

Para desarrollar la propuesta, seguimos las nueve etapas establecidas en la actividad. Estas incluyen el análisis inicial del caso, la selección de un patrón para organizar la lógica de negocio, la identificación de los elementos del dominio, el diseño de la capa de servicios, la definición de los patrones Repository y Unit of Work, la separación de Commands y Queries, la evaluación de los principios SOLID, la elaboración de diagramas y registros de decisiones arquitectónicas (ADR), y finalmente, la validación de la solución propuesta.

A lo largo del proceso, utilizamos inteligencia artificial como herramienta de apoyo para generar ideas, analizar alternativas y revisar posibles soluciones. Sin embargo, cada resultado fue comparado con los requerimientos del negocio y las indicaciones de la guía de la actividad antes de incorporarlo al trabajo. Asimismo, documentamos los errores encontrados, los ajustes realizados y los acuerdos del equipo en la bitácora de uso de IA. En este informe se presentan los resultados obtenidos en cada etapa, junto con las evidencias correspondientes.

Preparación

En esta etapa convertimos el material de la Unidad 2 en la base de trabajo para la arquitectura del sistema de Sazón Urbano. El BRD continúa siendo la guía principal, por lo que se conserva el alcance aprobado y las reglas de negocio establecidas, sin agregar condiciones que no estén documentadas. A partir de este material, se definieron cinco casos de uso que representan las actividades principales del proceso, incluyendo el registro de pedidos, la consulta de inventario, la preparación, las cancelaciones y el registro de pagos. También se elaboró una tabla de hallazgos para identificar inconsistencias, información incompleta y problemas detectados, indicando su tratamiento y estado.

Proceso principal: la gestión del pedido de Sazón Urbano, desde que el cliente realiza su solicitud por cualquiera de los canales de atención hasta que el pedido se entrega o se cancela. También se consideran los caminos de rechazo y las excepciones que pueden impedir que el proceso se complete correctamente.

1. Casos de uso

Seleccionamos cinco casos de uso del BRD, que incluyen operaciones normales y al menos un camino de rechazo cada uno, salvo el CU-04. Para cada caso documentamos el actor, el objetivo, la información de entrada, las reglas relacionadas, el resultado esperado y el camino de rechazo. Los cinco aparecen en las historias de usuario; el caso de promociones no aparece en el BPMN, por lo que se registra como hallazgo.

ID	Nombre	Actor	Objetivo	Información de entrada	Reglas relacionadas	Resultado esperado	Camino de rechazo
CU-01	Registrar pedido	Mesero/Caja	Registrar correctamente un pedido y enviarlo a cocina.	Canal, productos, cantidades, modificaciones y mesa o dirección de entrega.	RF-01, RF-02, RF-03. Solo se confirman productos disponibles y las modificaciones deben quedar registradas.	Pedido registrado con número de orden, estado correspondiente y envío automático a cocina.	Si un producto está agotado, el sistema informa la falta de disponibilidad y propone alternativas.
CU-02	Consultar inventario	Mesero, Encargado	Verificar la disponibilidad de productos antes de aceptar un pedido.	Nombre del producto o platillo solicitado.	RF-02, RF-04, RNF-01. La disponibilidad debe consultarse en tiempo real.	El sistema muestra si el producto está disponible y, cuando corresponda, los ingredientes faltantes.	Si no hay existencias suficientes, se informa que el producto no está disponible y se sugieren alternativas.
CU-03	Cancelar pedido	Mesero, Encargado, Gerente	Cancelar un pedido respetando las autorizaciones según su estado.	Número de pedido, usuario solicitante y motivo de cancelación.	RF-05. El mesero puede cancelar antes de iniciar la preparación; durante la preparación se requiere autorización del encargado o gerente.	Pedido cancelado, motivo registrado en bitácora, cocina notificada e inventario actualizado según corresponda.	Si el usuario no tiene autorización o no se aprueba la cancelación, el sistema rechaza la operación y conserva el pedido en su estado actual.

CU-04	Registrar pago	Mesero/Caja; repartidor para cobro contra entrega en efectivo	Registrar el pago de un pedido para facilitar el control y el cierre de caja.	Número de pedido, monto total, método de pago, monto recibido y referencia de transferencia cuando aplique.	RF-12, RF-13, RF-14, RF-15. Los pagos se registran por método; las transferencias requieren confirmación y no se almacenan datos de tarjetas.	Pago registrado con su estado, asociado al pedido y disponible para la conciliación de caja.	Si el pago es insuficiente, la transferencia no se confirma o el pago con tarjeta es rechazado, el sistema no lo marca como pagado y solicita corregir o verificar la operación.
CU-05	Registrar merma de inventario	Encargado	Registrar ingredientes dañados, caducados o extraviados para mantener la precisión del inventario.	Ingrediente, cantidad, motivo, responsable y evidencia fotográfica conforme al requisito de registro de mermas.	RF-09. Las mermas deben registrar causa y responsable; el inventario debe reflejar la pérdida.	Merma registrada en bitácora, existencias actualizadas y costo de pérdida calculado.	Si faltan datos obligatorios, la cantidad no es válida o no se identifica el ingrediente, el sistema rechaza el registro y solicita corregirlo.

2. Tabla de hallazgos

Se registraron doce hallazgos. Entre los más importantes están la separación de los roles Mesero y Caja, los permisos vacíos del Cliente en US-19, la necesidad de tratar la eliminación de una línea como cancelación parcial y las omisiones del BPMN respecto a promociones y alertas. Los tratamientos y estados siguientes son propuestas para su seguimiento.

ID	Hallazgo	Tratamiento	Estado
H-01	Los pedidos se registran en formatos distintos según el canal de atención.	Implementar una interfaz unificada para registrar pedidos de mesa, teléfono, WhatsApp y para llevar.	Propuesto
H-02	Las modificaciones de los pedidos se anotan en papel y pueden perderse o interpretarse incorrectamente.	Incorporar campos estructurados para modificaciones, validación y registro del usuario que realiza cada cambio.	Propuesto
H-03	El inventario se actualiza al final del día, lo que provoca diferencias y productos agotados sin detectar.	Implementar actualización del inventario en tiempo real, alertas de stock bajo y registro de mermas.	Propuesto
H-04	Los permisos de los empleados no están claramente definidos.	Establecer permisos por rol y reglas de autorización para modificaciones, cancelaciones, descuentos y consultas de reportes.	Pendiente de validación
H-05	Los pagos y las propinas no cuentan con un registro uniforme, lo que dificulta la conciliación de caja.	Implementar registro de pagos por método, control de propinas y reportes automáticos de cierre de caja.	Propuesto

H-06	No existe un protocolo digital para fallas de energía o conexión.	Implementar un modo offline que permita continuar operaciones y sincronizar registros al recuperar la conexión.	Pendiente de validación
H-07	Las quejas de los clientes no se registran de forma sistemática.	Crear un módulo para registrar quejas, categorías, soluciones y seguimiento.	Propuesto
H-08	No existe una base de datos completa de clientes ni un historial de sus pedidos.	Implementar un registro de clientes con datos de contacto, historial de compras y preferencias.	Propuesto

Selección del patrón de lógica de negocio

Para el sistema de Sazón Urbano se eligió Domain Model como patrón principal, utilizando Transaction Script únicamente para la operación de marcar un producto como agotado. Por su parte, Table Module se descarta para los cinco casos de uso analizados, aunque se mantiene como candidato para implementar los reportes del sistema.

La decisión se basa principalmente en la complejidad de las reglas del negocio y en la relación que existe entre los pedidos, el inventario y las estaciones de cocina. En primer lugar, varias reglas dependen del estado del pedido: según RN-02, la cancelación sin autorización solo es posible si ninguna estación ha iniciado la preparación; de lo contrario, conforme a RN-03, se requiere la autorización del Encargado y se debe generar el registro de la merma correspondiente. Domain Model permite centralizar estas reglas y controlar las transiciones de estado, evitando que se repitan en diferentes operaciones.

En segundo lugar, las mismas reglas deben aplicarse de manera consistente en los tres canales de pedido: mesa, para llevar y domicilio. Además, las promociones pueden cambiar con frecuencia, por lo que conviene mantener sus condiciones y validaciones en un modelo de dominio que facilite su modificación sin afectar innecesariamente otros procesos.

El principal costo de esta decisión es que Domain Model requiere una mayor estructura inicial que Transaction Script. Para evitar una complejidad excesiva, se propone trabajar con pocos aggregates y no convertir cada tabla de la base de datos en una entidad del dominio.

Finalmente, Transaction Script se reserva para marcar un producto como agotado, ya que se considera una operación lineal que no requiere administrar un ciclo de vida complejo. Table Module, en cambio, se considera más adecuado para una etapa posterior, especialmente para los reportes de ventas, inventario y cierre de caja, donde es necesario procesar y agrupar conjuntos de datos.

Tabla comparativa de patrones

Criterio	Transaction Script	Table Module	Domain Model

Complejidad de reglas	Baja a media. La lógica se organiza como procedimientos para completar una operación concreta.	Baja a media. Agrupa la lógica que opera sobre todos los registros de una tabla o conjunto relacionado.	Media a alta. Modela conceptos del negocio con comportamiento y reglas propias.
Cambios esperados	Adecuado si el flujo es estable y las reglas cambian poco; cambios frecuentes pueden multiplicar scripts.	Conveniente cuando cambian consultas, cálculos o procesos centrados en una tabla o reporte.	Conveniente si las reglas del negocio evolucionan y deben mantenerse consistentes en varios procesos.
Interacción entre conceptos	Limitada; suele coordinar pasos de una transacción y llamar servicios o repositorios.	Moderada, aunque normalmente se centra en una tabla o grupo de datos relacionado.	Alta; representa relaciones entre objetos como Pedido, Producto, Ingrediente, Pago y Cliente.
Estados e invariantes	Se validan dentro de cada script; existe riesgo de duplicar validaciones entre operaciones.	Puede aplicar validaciones sobre los datos de una tabla, pero no siempre representa bien invariantes entre entidades.	Es su punto fuerte: puede proteger transiciones válidas y reglas que siempre deben cumplirse.
Ventajas	Simple, rápido de implementar y fácil de entender para operaciones directas.	Mantiene junta la lógica de un conjunto de registros; útil para reportes y cálculos tabulares.	Organiza reglas complejas en el modelo del negocio y reduce la duplicación de reglas compartidas.
Riesgos	Scripts grandes o duplicados; las reglas pueden quedar dispersas y ser inconsistentes.	Puede convertirse en una capa procedimental extensa; las reglas entre varias entidades quedan menos claras.	Mayor costo inicial de diseño y aprendizaje; puede ser excesivo para funciones simples.
Consecuencias de elegirlo	Entrega inicial ágil, pero requiere disciplina para evitar que las reglas crezcan desordenadamente.	Facilita mantener reportes y operaciones por lote, aunque los procesos centrales pueden quedarse cortos.	Mejor consistencia para procesos centrales, a cambio de más diseño, pruebas y estructura.

Decisión para Sazón Urbano y evidencia del BRD

Transaction Script: operaciones sencillas y flujos directos

Decisión: utilizar Transaction Script para operaciones acotadas que ejecutan una secuencia clara de pasos y tienen reglas relativamente sencillas. Por ejemplo, registrar una solicitud de factura asociada a un ticket, enviar una notificación de pedido listo o registrar una queja con sus datos básicos.

Evidencia del BRD: RF-11 solicita notificaciones por eventos relevantes; RF-14 contempla generar tickets digitales y registrar solicitudes de factura; RF-17 pide registrar quejas, categoría, resolución y seguimiento. Son operaciones identificables con entradas y resultados concretos. La historia 15 también muestra una notificación al mesero cuando el pedido está listo.

Ventajas: es sencillo de implementar dentro del plazo de ocho semanas y permite probar cada flujo de manera independiente. Riesgos: no conviene repetir en varios scripts reglas críticas de pedidos, pagos o inventario. Si una regla se comparte, debe centralizarse para evitar resultados diferentes según el punto desde el que se ejecute la operación.

Table Module: reportes y cálculos centrados en conjuntos de datos

Decisión: usar Table Module para reportes y cálculos que trabajan principalmente sobre un conjunto de registros, como reportes de ventas por hora, día y producto, métricas de errores y reportes básicos de inventario. Cada módulo puede reunir las consultas, agrupaciones y cálculos relacionados con el conjunto de datos que reporta.

Evidencia del BRD: RF-08 requiere reportes de ventas, inventario y errores; RF-15 requiere un reporte de cierre de caja con conciliación por método de pago. La historia 11 presenta ventas totales, productos más vendidos, horas pico, duración promedio y errores, que son resultados agregados a partir de muchos pedidos. El alcance también incluye reportes operativos y el BRD aclara que no se construirá un sistema contable completo, sino reportes básicos.

Ventajas: concentra la lógica de consultas y agregaciones, facilitando modificar los reportes cuando gerencia solicite nuevas métricas. Riesgos: no debe convertirse en el lugar donde se decidan reglas centrales, como si un pedido puede cancelarse o si un pago está confirmado. La conciliación puede presentar diferencias y requiere definir claramente qué registros y estados se consideran en cada cálculo.

Domain Model: procesos centrales con reglas e invariantes

Decisión: usar Domain Model para los conceptos centrales del negocio, principalmente Pedido, Producto/Ingrediente, Inventario, Pago, Promoción y Cliente/Lealtad. Estos conceptos tienen relaciones, estados y reglas que deben conservarse consistentes a través de distintas operaciones.

Evidencia del BRD: RF-01 registra pedidos con productos, cantidades y modificaciones; RF-02 consulta disponibilidad en tiempo real; RF-04 actualiza inventario; RF-05 contempla cancelaciones con autorización; RF-06 administra promociones; RF-12 registra pagos; RF-18 gestiona puntos y recompensas. Las historias 1, 5, 9, 10, 16 y 18 muestran reglas que interactúan: un pedido se confirma, pasa a preparación, puede requerir autorización para cancelarse, descuenta ingredientes, valida promociones contra stock y acumula puntos al completarse el pago.

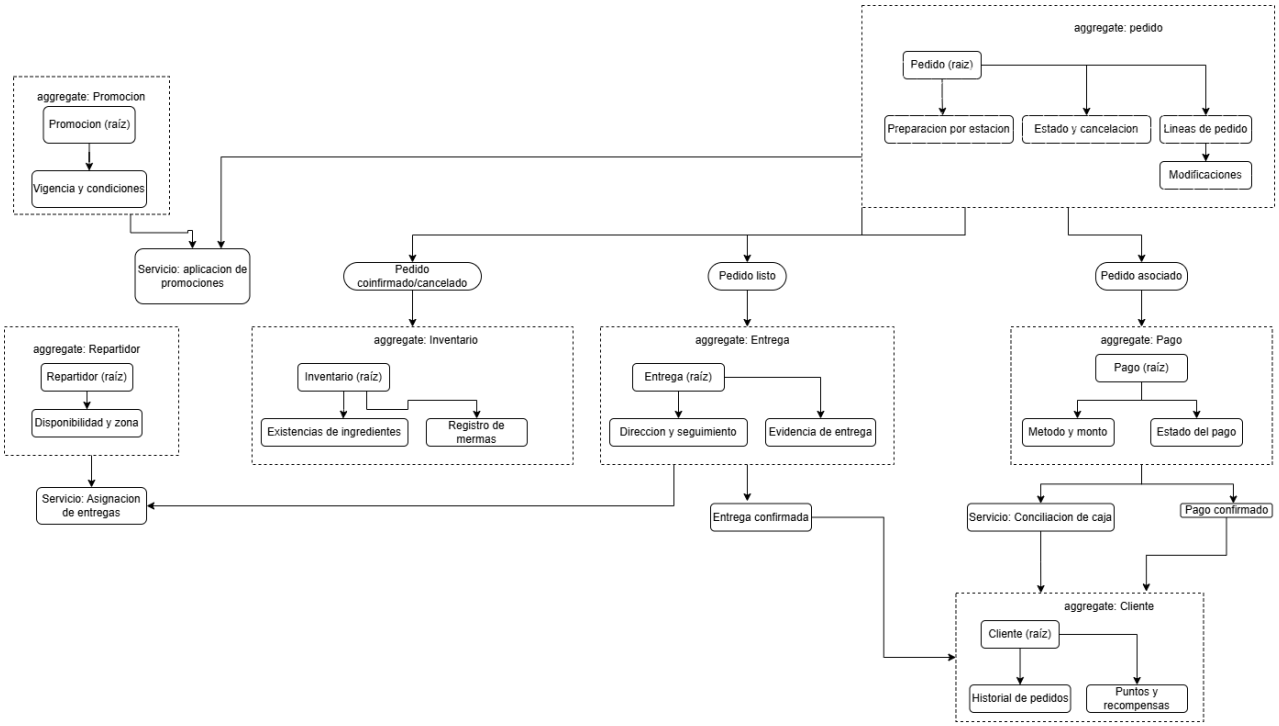
Invariantes que debería proteger el modelo: no confirmar un pedido con productos no disponibles; no permitir que un usuario sin permiso cancele un pedido que ya está en preparación; registrar los cambios de estado y su responsable; descontar o liberar inventario de acuerdo con la transición válida del pedido; no acumular puntos antes de que el pago cumpla la condición definida; y no marcar una transferencia como confirmada sin verificación. Estas reglas se derivan de los flujos y requerimientos del BRD y deberán precisarse durante el diseño.

Ventajas: evita duplicar reglas críticas y ayuda a mantener consistencia entre pedidos, inventario, pagos y lealtad. Riesgos: requiere más diseño que un script sencillo y puede complicar el proyecto si se modela cada detalle desde el inicio. Para controlar el alcance, conviene empezar por las entidades y reglas necesarias para los requerimientos Must Have.

Resumen de asignación

Patrón	Uso propuesto	Evidencia del BRD
Transaction Script	Notificaciones, registro de quejas y solicitud de factura; flujos cortos y concretos.	RF-11, RF-14, RF-17; historia 15.
Table Module	Reportes de ventas, inventario, errores y agregados de cierre de caja.	RF-08 y RF-15; historia 11.
Domain Model	Pedido, inventario, pagos, promociones, cancelaciones y puntos de lealtad.	RF-01, RF-02, RF-04, RF-05, RF-06, RF-12, RF-18; historias 1, 5, 9, 10, 16 y 18.

Diagrama



Obtuvimos lo siguiente:

- Siete aggregates: Pedido, Promoción, Inventario, Entrega, Repartidor, Pago y Cliente. Se conocen entre sí solo por identificador.

- Value objects: Canal, Dinero, Cantidad, Modificación, Vigencia, Estación de cocina, Método de pago, Estado de pago, Dirección de entrega, Evidencia de entrega, Zona, Responsable.
- Tres domain services: AplicadorDePromocion (coordina Pedido y Promoción), AsignadorDeEntregas (coordina Entrega y Repartidor) y ConciliadorDeCaja (coordina los Pagos del turno).
- Domain events: PedidoConfirmado, PedidoCancelado, PedidoEnPreparacion, PedidoListo, EntregaConfirmada, PagoConfirmado, StockBajo, MermaRegistrada.

Ficha de aggregates

Pedido

Campo	Contenido
Raíz	Pedido
Contiene	Líneas de pedido con sus modificaciones, preparación por estación, estado y cancelación, canal, responsable
Responsabilidades	Confirmarse, modificarse, cancelarse (total o por línea), recalcular su total, avanzar de estado, marcarse listo
Invariantes	No se confirma con productos sin stock (RN-01) · Domicilio exige teléfono y dirección (RN-02) · Cancelar antes de preparación no requiere autorización (RN-03) · Cancelar en preparación exige autorizar al Encargado y registrar motivo (RN-04) · Toda modificación registra usuario y fecha-hora (RN-05) · Una sola promoción por pedido (RN-06) · El total coincide con líneas y descuento
Estados	Registrado → Confirmado → En preparación → Listo → Entregado, o Cancelado
Eventos	PedidoConfirmado, PedidoCancelado, PedidoEnPreparacion, PedidoListo
Casos de uso	Registrar pedido, aplicar promoción, cancelar pedido

Promoción

Campo	Contenido

Raíz	Promoción
Contiene	Vigencia (días y horario), productos aplicables, condiciones, regla de descuento
Responsabilidades	Decir si es aplicable en una fecha-hora y para ciertos productos; calcular su descuento
Invariantes	Siempre tiene días, horario y productos definidos (RN-07) · Solo el Gerente la crea o modifica; es un permiso de la capa de aplicación (RN-08) · No es aplicable si el stock de sus productos es insuficiente (RN-09)
Eventos	Ninguno en los casos elegidos
Casos de uso	Aplicar promoción

Inventario

Campo	Contenido
Raíz	Inventario
Contiene	Existencias por ingrediente (con punto de reorden) y registro de mermas
Responsabilidades	Descontar ingredientes al preparar, devolverlos al cancelar sin preparación, registrar mermas, avisar stock bajo
Invariantes	Las existencias nunca son negativas · Se descuenta al pasar el pedido a "En preparación" (RN-10) · Una merma exige ingrediente, cantidad, motivo y responsable (RN-11) · Una merma registrada no se edita (trazabilidad) · Al bajar del mínimo se genera alerta
Eventos	StockBajo, MermaRegistrada
Casos de uso	Cancelar pedido (libera o genera merma), aplicar promoción (consulta stock)

Entrega

Campo	Contenido
Raíz	Entrega (referencia PedidoId y RepartidorId)
Contiene	Dirección y seguimiento (estado, hora de salida y de entrega), evidencia de entrega
Responsabilidades	Asignarse, iniciar ruta, confirmarse o fallar
Invariantes	Solo existe para pedidos a domicilio en estado Listo (RN-12) · Confirmar exige foto y firma (RN-13) · No registra entrega sin hora de salida · Una entrega fallida exige motivo
Eventos	RepartidorAsignado, EntregaConfirmada, EntregaFallida
Casos de uso	Asignar repartidor y confirmar entrega

Repartidor

Campo	Contenido
Raíz	Repartidor
Contiene	Disponibilidad y zona
Responsabilidades	Conocer si está disponible y a qué zona pertenece; marcarse ocupado o libre
Invariantes	No se asigna a uno no disponible (RN-14) · Solo recibe pedidos de su zona
Eventos	Ninguno
Casos de uso	Asignar repartidor

Pago

Campo	Contenido

Raíz	Pago (referencia PedidoId)
Contiene	Método y monto, estado del pago, propina
Responsabilidades	Registrarse, confirmarse, calcular cambio en efectivo
Invariantes	Transferencia nace "Pendiente de confirmación" hasta que el Encargado la verifica (RN-15) · Tarjeta se registra sin almacenar datos de la tarjeta (RN-16) · El monto registrado coincide con el total del pedido · Efectivo exige monto recibido ≥ total
Eventos	PagoConfirmado
Casos de uso	Registrar pago

Cliente

Campo	Contenido
Raíz	Cliente
Contiene	Datos de contacto, historial de pedidos (solo referencias por ID), puntos y recompensas
Responsabilidades	Acumular puntos, canjear recompensas, conservar preferencias
Invariantes	Se acumula 1 punto por cada $10 pagados (RN-17) · No se canjea con puntos insuficientes · Los puntos solo se acreditan con un pago confirmado
Eventos	Ninguno
Casos de uso	Registrar pago (acredita puntos)

Service layer

El Service Layer es la puerta de entrada de los casos de uso. Funciona como el director de una orquesta: decide qué pasos se realizan y en qué orden, pero no contiene reglas de negocio. En esta etapa definimos los Application Services,

qué recibe y devuelve cada uno, con quién colabora y qué reglas delega al dominio.

Diseñamos cinco métodos, uno por caso de uso, agrupados en tres servicios: **PedidoApplicationService** (registrar pedido, aplicar promoción y cancelar pedido), **PagoApplicationService** (registrar pago) y **EntregaApplicationService** (gestionar entrega). Separamos los servicios por responsabilidad: los cobros y las entregas no se mezclan con la gestión de pedidos.

Los dos métodos que cubren varios pasos (registrarPago y gestionarEntrega) reciben una solicitud que indica cuál se pide y la enrutan al comportamiento correcto; enrutar no es una decisión de negocio.

Para cada método justificamos dónde vive cada regla. Por ejemplo, la autorización de cancelaciones, la exigencia de teléfono y dirección en domicilio y la regla de una sola promoción viven en el Pedido; la vigencia y los productos de una promoción viven en la Promoción; el estado “pendiente de confirmación” de una transferencia vive en el Pago; y el control de qué rol puede hacer cada acción vive en la capa de aplicación, porque es seguridad por rol y no una regla de dominio.

Tabla de application services

Servicio	Método (CU)	Entrada	Colaboradores
PedidoApplicationService	registrarPedido (CU-01)	Canal, líneas (producto, cantidad, modificaciones), datos del cliente, mesa, responsable	Control de acceso, PedidoRepository, ConsultasDeInventario (solo lectura), Unit of Work, publicador de eventos
PedidoApplicationService	aplicarPromocion (CU-02)	Id del pedido, id de la promoción elegida, fecha-hora, responsable	Control de acceso, PedidoRepository, PromocionRepository, ConsultasDeInventario, AplicadorDePromocion, Unit of Work
PedidoApplicationService	cancelarPedido (CU-03)	Id del pedido, motivo, solicitante, autorizador (Encargado, solo	Control de acceso, PedidoRepository, InventarioRepository, Unit of Work, publicador

		si ya inició preparación)	
PagoApplicationService	registrarPago (CU-04)	Una de dos solicitudes (ver abajo) + responsable	Control de acceso, PedidoRepository, PagoRepository, ClienteRepository, Unit of Work, publicador
EntregaApplicationService	gestionarEntrega (CU-05)	Una de cuatro solicitudes (ver abajo) + responsable	Control de acceso, PedidoRepository, RepartidorRepository, EntregaRepository, AsignadorDeEntregas, Unit of Work, publicador

Tabla de solicitudes que enruta cada método

Método	Solicitud	Quién la pide	Datos	Efecto
registrarPago	Registrar pago	Caja	Id del pedido, método, monto (y monto recibido si es efectivo; referencia si es transferencia)	Pago asociado al pedido con su estado inicial
	Confirmar transferencia	Encargado	Id del pago	Pago confirmado y puntos acreditados al cliente
gestionarEntrega	Asignar repartidor	Encargado	Id del pedido, id del repartidor	Se crea la entrega vinculada al repartidor
	Registrar salida	Repartidor	Id de la entrega	Entrega en ruta, con hora de salida
	Confirmar entrega	Repartidor	Id de la entrega, foto, firma	Entrega confirmada y pedido entregado

	Registrar entrega fallida	Repartidor	Id de la entrega, motivo	Entrega fallida y pedido en “entrega no completada”

Repository y Unit of Work

El patrón Repository separa la lógica de negocio de la forma en que se guardan y recuperan los datos. El dominio pide “dame el pedido 1001” sin saber si detrás hay SQL, una API o archivos. La Unit of Work decide qué cambios se guardan juntos: o se confirma todo, o no se guarda nada. En esta etapa definimos ambas piezas.

Lo que hicimos fue:

- Siete repositorios con catorce métodos en total, uno por aggregate, todos usados por los cinco casos de uso. Las interfaces viven en el dominio y hablan en términos del negocio; las implementaciones viven en infraestructura (en memoria para el código base, y una relacional como posibilidad).
- Un contrato común: obtener por id siempre lanza un error de negocio si el objeto no existe, sin importar la implementación.
- Una unidad de trabajo por solicitud: inicia al comenzar el método del servicio, se confirma al final y, si algo falla, se deshace todo y no se publica ningún evento. Los eventos se publican solo después de confirmar.
- Identificamos qué debe guardarse junto: al cancelar con preparación iniciada, el Pedido y el Inventario (con su merma); al cancelar sin preparación, el Pedido y la devolución de ingredientes al Inventario; al confirmar una transferencia, el Pago y el Cliente (puntos); al cerrar una entrega, la Entrega, el Pedido y el Repartidor.
- Distinguimos errores de negocio (por ejemplo, producto agotado o cancelación sin autorización) de errores técnicos (por ejemplo, una caída de conexión, algo que el BRD advierte por la red intermitente y por el WiFi básico del local).

Decidimos no implementar protección contra concurrencia para mantener el diseño simple; es un riesgo aceptado, que se documentará en los ADR.

Tabla de Repositorios

Repository	**Necesidad del dominio**	**Métodos propuestos**	**Objeto que recupera o guarda**

PedidoRepository	Recuperar un pedido completo para cancelarlo, aplicarle promoción o entregarlo. Guardar el pedido nuevo o modificado	obtenerPorId(pedidoId) · guardar(pedido)	Pedido completo (con líneas, modificaciones, preparación por estación y cancelación)
PromocionRepository	Conocer una promoción para saber si aplica	obtenerPorId(promocionId)	Promoción
InventarioRepository	Descontar o devolver ingredientes y registrar mermas al cancelar	obtener() · guardar(inventario)	Inventario (existencias y registro de mermas)
EntregaRepository	Recuperar una entrega en curso y guardar su avance	obtenerPorId(entregaId) · guardar(entrega)	Entrega
RepartidorRepository	Saber si un repartidor está disponible y actualizar su disponibilidad	obtenerPorId(repartidorId) · guardar(repartidor)	Repartidor
PagoRepository	Registrar un pago y recuperarlo para confirmar una transferencia	obtenerPorId(pagoId) · guardar(pago)	Pago
ClienteRepository	Acreditar puntos tras un pago confirmado	obtenerPorId(clienteId) · guardar(cliente)	Cliente (con puntos; historial solo por referencias)

Tabla de Unit of Work

Método de servicio	**Cambios que deben completarse juntos**	**Si algo falla**
registrarPedido	Pedido nuevo (confirmado, con sus líneas)	No se guarda nada y **no se envía a cocina**

aplicarPromocion	Pedido con promoción y total recalculado	El pedido queda como estaba
cancelarPedido	Pedido cancelado + Inventario (ingredientes devueltos si no había preparación, o merma registrada si ya la había)	Ni cancelación ni ajuste de inventario: nunca queda un pedido cancelado sin su merma o su devolución (RN-04, RN-11)
registrarPago	Registrar: Pago nuevo · Confirmar transferencia: Pago confirmado + Cliente con puntos acreditados	Se deshace todo y **no se publica PagoConfirmado**
gestionarEntrega	Asignar: Entrega nueva + Repartidor no disponible · Salida: Entrega · Confirmar o fallar: Entrega + Pedido + Repartidor disponible	Se deshace todo y la entrega queda en su estado anterior

Commands y Queries: CQRS básico

CQRS (Command Query Responsibility Segregation) propone separar las operaciones que cambian el sistema (commands) de las que solo consultan información (queries). Un command ordena que algo cambie y aplica reglas; un query pregunta cómo está algo y nunca modifica nada. En esta etapa aplicamos una versión básica de esa separación.

Lo que hicimos fue lo siguiente:

- **Nueve commands**, uno por cada operación que cambia el estado, cada uno con su handler: RegistrarPedido, AplicarPromocion, CancelarPedido, RegistrarPago, ConfirmarTransferencia, AsignarRepartidor, RegistrarSalidaDeEntrega, ConfirmarEntrega y RegistrarEntregaFallida. Los cinco métodos del Service Layer solo enrutan la solicitud al handler correcto; las reglas viven en el dominio.
- **Cuatro queries:** resumen del pedido, menú disponible, historial del cliente y cierre de caja. Dependen de interfaces de lectura (ConsultasDePedidos, ConsultasDeInventario, ConsultasDeClientes y ConsultasDeCaja) y devuelven DTO (objetos simples que solo transportan datos), nunca el aggregate.
- **Una sola base de datos:** la separación es en el código. Separar almacenes de lectura y escritura sería complejidad sin beneficio para un restaurante único, con 80 a 150 pedidos diarios.

- **El cierre de caja** es una lectura agregada por método de pago, sin reglas de negocio propias. Por eso es el mejor candidato para un Table Module o para un modelo de lectura más pesado si el sistema crece.
- **Lo agotado deja de mostrarse** mediante la query del menú disponible, sin agregar una regla extra al command RegistrarPedido. El command sigue validando el stock por su cuenta, porque el menú puede estar desactualizado al momento de confirmar.

Un **DTO** (Data Transfer Object) es lo que devuelven las queries (por ejemplo, ResumenDePedidoDto). Por eso no usamos el aggregate Pedido para mostrar datos: el aggregate protege reglas y el DTO solo muestra información, sin exponer detalles internos como la preparación por estación ni las cancelaciones.

Tabla de Commands

Tipo	Nombre	Datos de entrada	Handler y dependencias	Resultado	Justificación
Command	RegistrarPedido	Canal, líneas, datos del cliente, mesa, responsable	RegistrarPedidoHandler → PedidoRepository, ConsultasDeInventario, Unit of Work, publicador	Id, total y estado del pedido	Crea el pedido y aplica RN-01, RN-02 y RN-05 (CU-01)
Command	AplicarPromocion	Id del pedido, id de la promoción, fecha-hora, responsable	AplicarPromocionHandler → PedidoRepository, PromocionRepository, ConsultasDeInventario, AplicadorDePromocion, Unit of Work	Pedido con descuento y total recalculado	Cambia el total y exige una sola promoción con stock suficiente (RN-06, RN-09)
Command	CancelarPedido	Id del pedido, motivo, solicitante, autorizador (si aplica)	CancelarPedidoHandler → PedidoRepository, InventarioRepository, Unit of Work, publicador	Cancelación confirmada, indicando si liberó inventario o generó merma	Aplica RN-03, RN-04 y RN-11

Command	RegistrarPago	Id del pedido, método, monto (monto recibido o referencia según el método), responsable	RegistrarPagoHandler → PedidoRepository, PagoRepository, Unit of Work, publicador	Pago registrado con su estado (Pagado, Aprobado o Pendiente)	Aplica RN-15 y RN-16; el monto debe coincidir con el total (CU-04)
Command	ConfirmarTransferencia	Id del pago, responsable (Encargado)	ConfirmarTransferenciaHandler → PagoRepository, ClienteRepository, Unit of Work, publicador	Pago confirmado y puntos acreditados	Cambia el estado del pago y dispara PagoConfirmado (RN-15, RN-17)
Command	AsignarRepartidor	Id del pedido, id del repartidor, responsable	AsignarRepartidorHandler → PedidoRepository, RepartidorRepository, EntregaRepository, AsignadorDeEntregas, Unit of Work	Entrega con repartidor asignado	Crea la entrega y marca al repartidor no disponible (RN-12, RN-14)
Command	RegistrarSalidaDeEntrega	Id de la entrega, responsable	RegistrarSalidaHandler → EntregaRepository, Unit of Work	Entrega en ruta, con hora de salida	Registra la hora de salida exigida antes de confirmar
Command	ConfirmarEntrega	Id de la entrega, foto, firma, responsable	ConfirmarEntregaHandler → EntregaRepository, PedidoRepository, RepartidorRepository, Unit of Work, publicador	Entrega confirmada y pedido entregado	Aplica RN-13 y cierra el pedido
Command	RegistrarEntregaFallida	Id de la entrega, motivo, responsable	RegistrarEntregaFallidaHandler → mismas dependencias que ConfirmarEntrega	Pedido en “entrega no completada”	El motivo es obligatorio

Tabla de Queries

Tipo	Nombre	Datos de entrada	Handler y dependencias	Resultado	Justificación
Query	ConsultarResumenDelPedido	Id del pedido	ConsultarResumenHandler → ConsultasDePedidos	ResumenDePedidoDto	Mesero y Caja revisan el pedido antes de aplicar promoción, cobrar o cancelar
Query	ConsultarMenuDisponible	Fecha-hora	ConsultarMenuHandler → ConsultasDeInventario	Lista de ProductoDisponibleDto	Lo agotado deja de mostrarse sin agregar una regla extra a RegistrarPedido (Historia 2)
Query	ConsultarHistorialDelCliente	Teléfono o id del cliente	ConsultarHistorialHandler → ConsultasDeClientes	HistorialDeClienteDto	El mesero ofrece recomendaciones y el cajero ve los puntos (Historias 13 y 18)
Query	ConsultarCierreDeCaja	Fecha, turno	ConsultarCierreHandler → ConsultasDeCaja	CierreDeCajaDto	Compara ingresos esperados contra reales por método de pago; es lectura agregada (RF-15)

Tabla de DTO (modelo de lectura)

DTO	Campos	Por qué no es el modelo de escritura
ResumenDePedidoDto	Id, canal, mesa o cliente (texto), líneas (producto, cantidad, modificaciones, subtotal), promoción (nombre y descuento), total, estado (texto), responsable (nombre)	Es plano y está listo para mostrar: no tiene métodos ni reglas, ni expone la preparación por estación ni la cancelación interna. El aggregate Pedido protege reglas; este DTO solo muestra datos
ProductoDisponibleDto	Id, nombre, precio, disponible (sí/no)	El inventario no se expone como aggregate a la pantalla; solo se muestra lo necesario para ofrecer el producto
HistorialDeClienteDto	Id, nombre, teléfono, últimos pedidos (fecha, resumen, total), preferencias, puntos acumulados	Junta datos de Cliente y Pedido sin romper la regla de que Cliente solo guarda referencias por ID
CierreDeCajaDto	Fecha, turno, total esperado y total real por método (efectivo, tarjeta, transferencia), diferencia, transferencias pendientes	Es un reporte calculado, no un objeto con reglas; se arma desde las lecturas de Pago

Revisión con SOLID

Antes de cerrar con los diagramas, pasamos el diseño por cada principio SOLID y lo contrastamos con lo que ya teníamos escrito. Buscamos tres cosas: clases que cargaran con demasiado, dependencias que no hacían falta y reglas que nos obligaran a abrir código que ya funciona. Algunas decisiones salieron bien y otras las ajustamos. Las que decidimos no tocar quedan registradas como riesgo.

Tabla de revisión SOLID

Principio	Qué revisamos	Qué encontramos	Decisión
S Responsabilidad única	Servicios, handlers y aggregates	Pedido, Pago y Entrega tienen su propio servicio, y los métodos multipaso solo derivan la solicitud al handler correspondiente. El aggregate Inventario junta existencias, mermas y alertas	Se mantiene la separación de servicios. Inventario queda como riesgo aceptado

O Abierto/cerrado	Reglas que podrían crecer: pagos, promociones y lealtad	Promoción ya calcula su propio descuento. En Pago, el estado inicial dependía del método con condiciones encadenadas. Los puntos de lealtad estaban fijos en "1 por cada $10"	Se corrigen Pago y lealtad
L Sustitución de Liskov	Repositorios en memoria frente a relacionales	El contrato cubría los repositorios con id, pero InventarioRepository.obtener() no tenía definido qué pasa en el primer arranque	Se amplía el contrato
I Segregación de interfaces	Qué necesita realmente cada consumidor	Los handlers de pedido y promoción solo preguntan si hay stock, y las queries usan interfaces de lectura	Se mantiene
D Inversión de dependencias	Dirección de las dependencias	Dominio y aplicación solo conocen interfaces, y el Pedido recibe la disponibilidad ya consultada en lugar de pedirla al inventario	Se mantiene

Detalle de lo que ajustamos

Pago y métodos de cobro. Efectivo exige monto recibido y calcula cambio, tarjeta se registra como aprobada sin guardar datos de la tarjeta, y transferencia queda pendiente hasta que el Encargado la verifica. Si todo eso vivía en una cadena de condiciones dentro de Pago, un método nuevo (por ejemplo QR) obligaría a editar esa clase. Ahora cada método es una variante que define su estado inicial y sus validaciones, y Pago no cambia cuando aparece otra.

Puntos de lealtad. La Historia 18 y el RF-18 hablan de un programa que el gerente configura, pero la regla estaba escrita dentro de Cliente. Pasó a ser una política de lealtad vigente que Cliente solo aplica, sin conocer el valor.

Contrato del inventario. Como obtener() no recibe id, definimos que siempre devuelve un inventario (vacío si aún no existe) o lanza un error de negocio. Así la versión en memoria y la relacional se comportan igual desde el primer uso.

Riesgos que dejamos documentados

- **Pedido concentra mucho comportamiento.** Maneja estado, líneas, promoción y cancelación. No lo partimos porque dividirlo rompería la

consistencia que protege como aggregate. Lo acotamos con value objects y con AplicadorDePromocion.

- **Inventario es grande.** Con más de 150 ingredientes, cargarlo completo puede pesar. Lo mantenemos unido porque cancelar un pedido debe devolver ingredientes o registrar la merma en la misma operación. Si el módulo crece, el primer candidato a separarse es Merma.

Cambios que arrastra a otras etapas

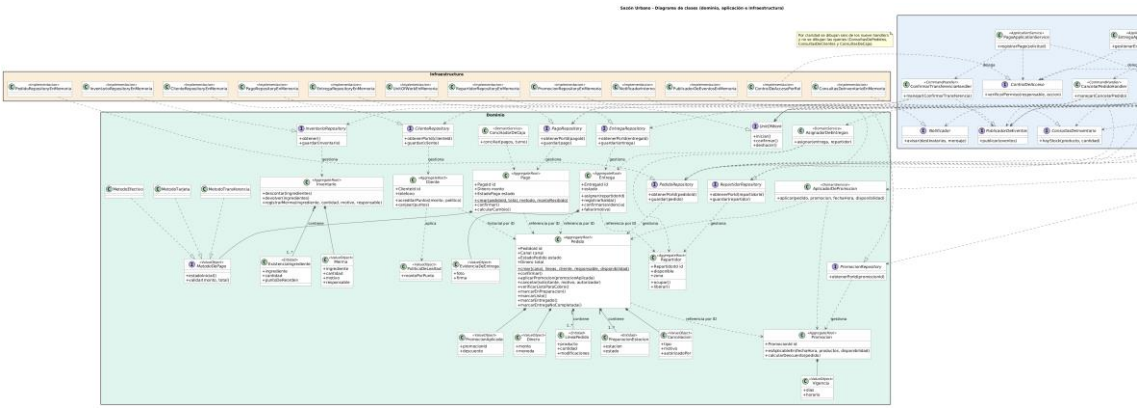
Las correcciones de Pago, lealtad y contrato de inventario modifican las fichas de Pago y Cliente (Etapa 3) y el contrato común (Etapa 5), y deben reflejarse también en el diagrama de clases y en la bitácora.

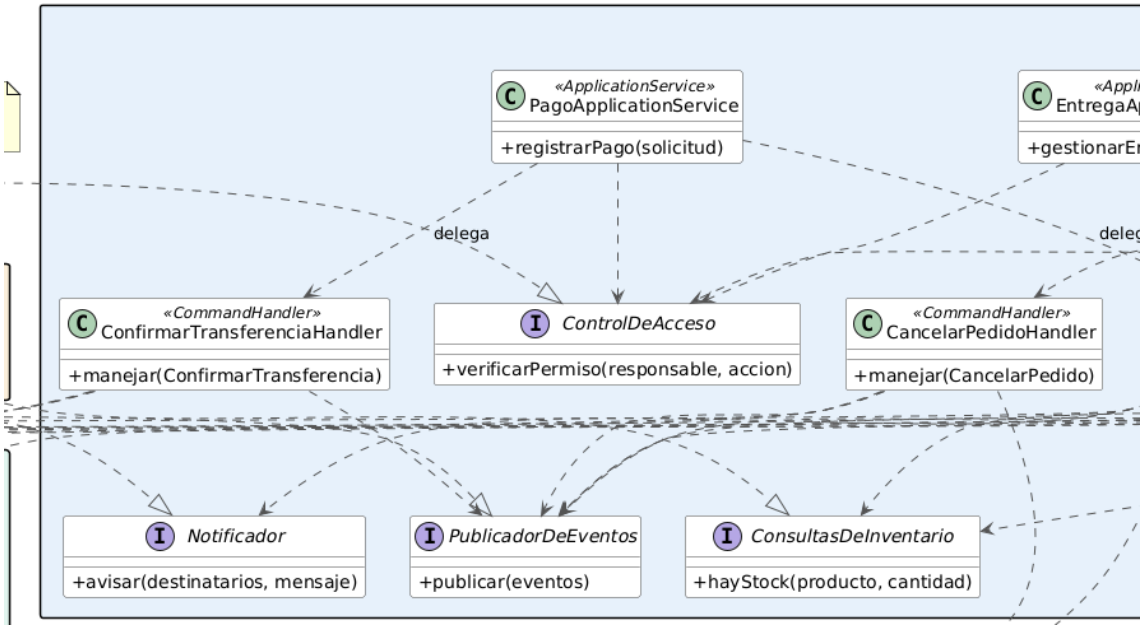
Diagrama UML

Elaboramos el diagrama de clases completo y dos diagramas de secuencia en PlantUML. Los archivos fuente (.puml) y sus exportaciones en PNG están en el repositorio. Comprobamos que las dos secuencias usen solo clases que existen en el diagrama de clases y que ninguna clase quede sin relaciones.

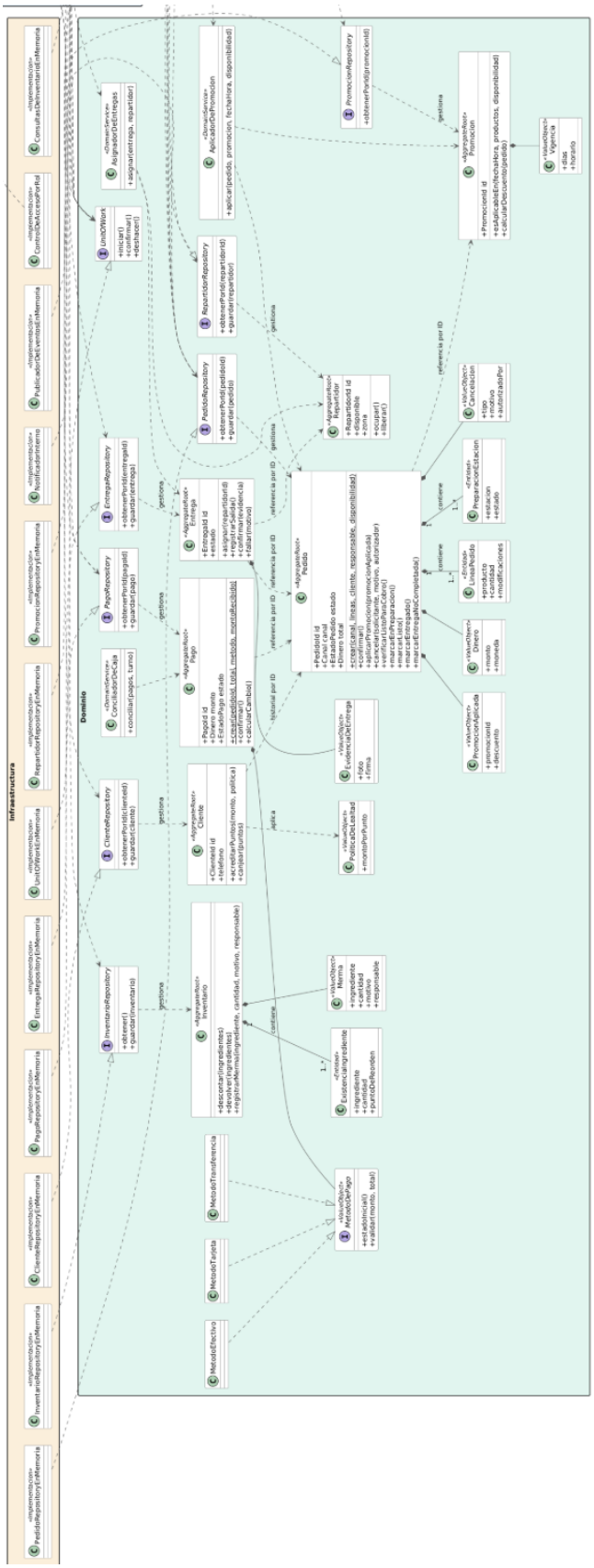
El diagrama de clases separa las tres capas por color: dominio (verde), aplicación (azul) e infraestructura (naranja). Las flechas de dependencia apuntan a interfaces y no a implementaciones. Por claridad se dibujan seis de los nueve handlers y no se dibujan las queries.

La primera secuencia muestra el flujo principal de registrar un pago en efectivo. La segunda muestra la aplicación de una promoción con tres alternativas: fuera de vigencia, stock insuficiente y pedido que ya tiene promoción.




«ApplicationService»
PagoApplicationService
+registrarPago(solicitud)
delega
«CommandHandler»
ConfirmarTransferenciaHandler
+manejar(ConfirmarTransferencia)
ControlDeAcceso
+verificarPermiso(responsable, accion)
«CommandHandler»
CancelarPedidoHandler
+manejar(CancelarPedido)
Notificador
+avisar(destinatarios, mensaje)
PublicadorDeEventos
+publicar(eventos)
ConsultasDeInventario
+hayStock(producto, cantidad)

Reporte



Código del diagrama de clases en plantuml

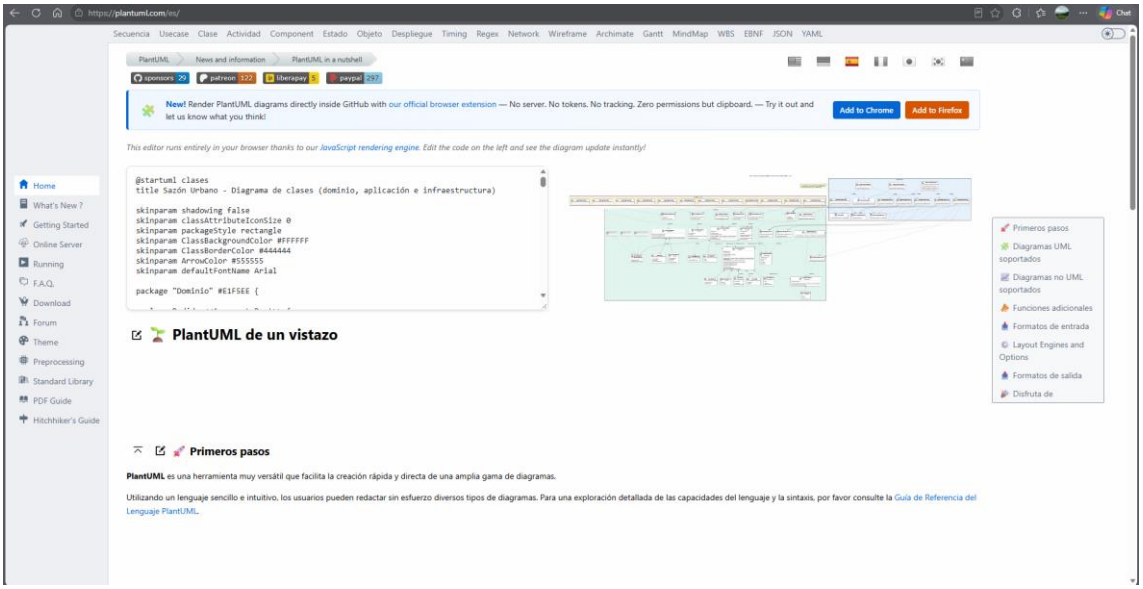
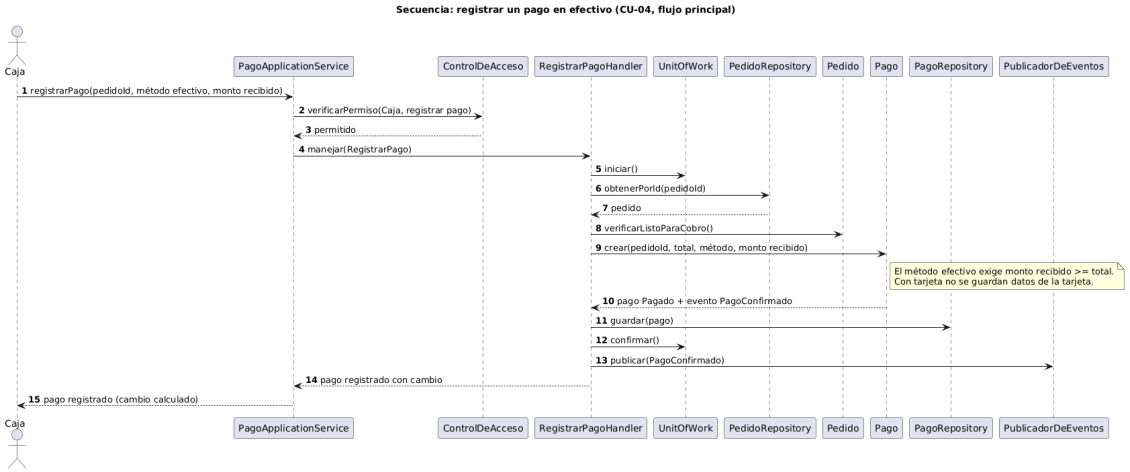
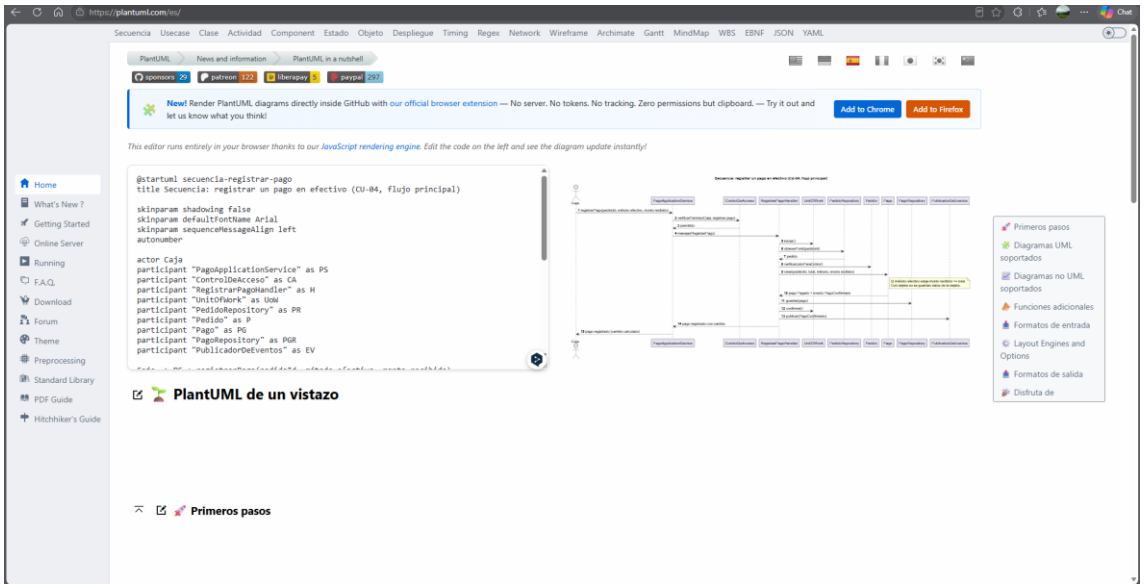


Diagrama de secuencia

Flujo principal (CU-04): registrar un pago en efectivo. Muestra cómo se orquesta la operación de punta a punta: la Caja envía la solicitud de cobro, el servicio revisa que su rol tenga permiso y le pasa el trabajo al handler, que abre la unidad de trabajo y recupera el pedido. El Pedido confirma que está listo para cobrarse y el Pago se crea aplicando sus propias reglas (el monto recibido en efectivo debe cubrir el total y se calcula el cambio). Después se guarda el pago, se confirma la unidad de trabajo y, solo entonces, se publica el evento `PagoConfirmado`. La Caja recibe el pago registrado con el cambio calculado.



Código del diagrama de secuencia del registro en plantuml

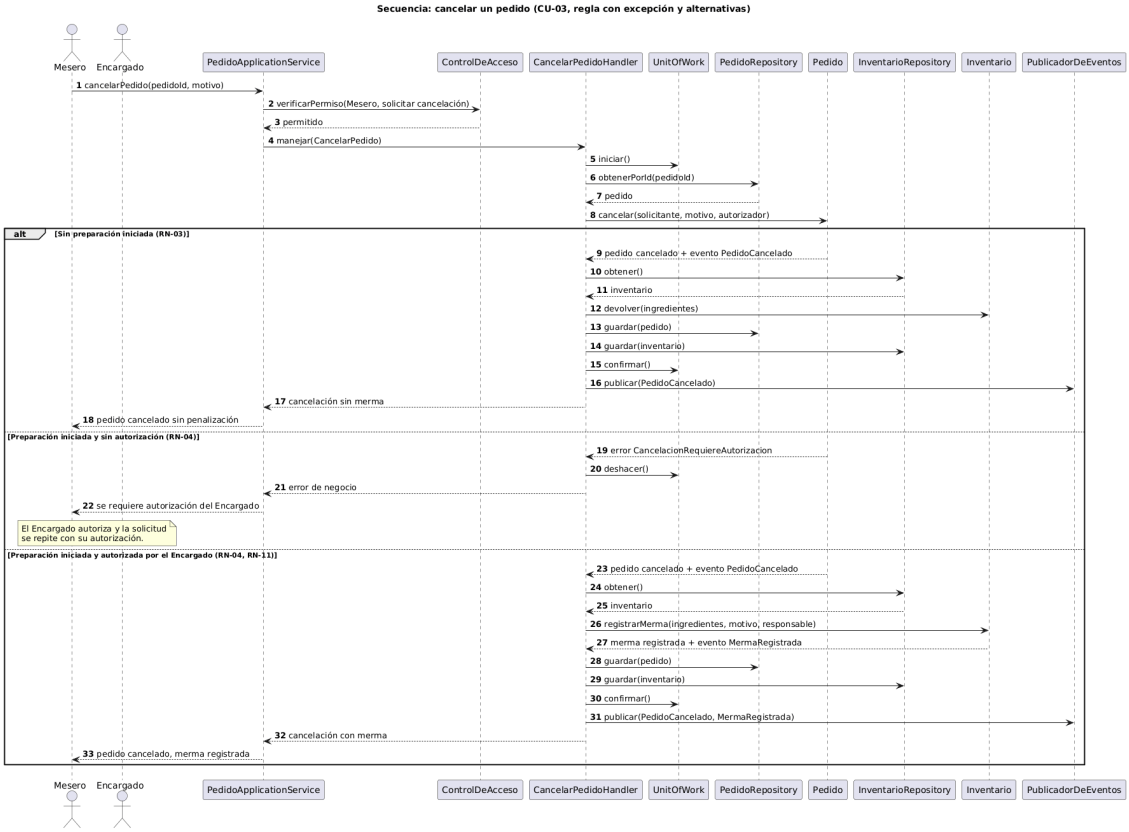


Regla con excepción y alternativas (CU-03): cancelar un pedido. El flujo sigue el mismo patrón de orquestación, pero el resultado depende de si cocina ya empezó a preparar. La solicitud la hace el Mesero o la Caja, el servicio verifica que su rol pueda solicitar cancelaciones, y el handler abre la unidad de trabajo, recupera el pedido y le pide cancelarse. A partir de ahí hay tres caminos:

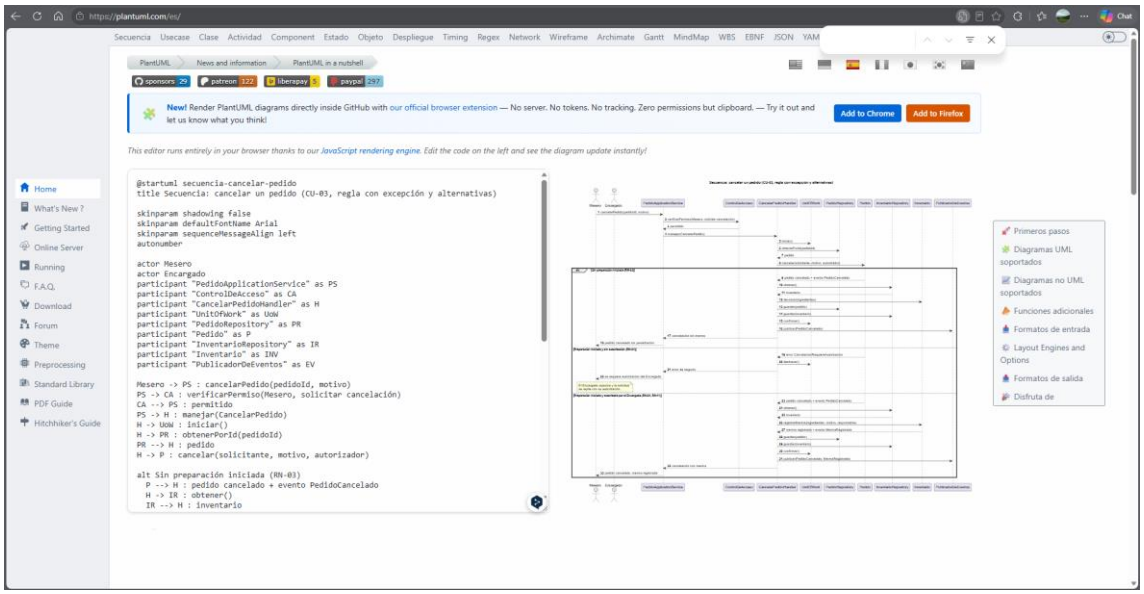
1. **Sin preparación iniciada:** el Pedido se cancela sin penalización y sin pedir autorización (RN-03). El handler devuelve al Inventario los ingredientes que se habían descontado, guarda ambos aggregates, confirma la unidad de trabajo y publica `PedidoCancelado`.
2. **Con preparación iniciada y sin autorización:** el Pedido rechaza la cancelación con el error `CancelacionRequiereAutorizacion` (RN-04). Se deshace la unidad de trabajo, no se guarda nada y no se publica ningún evento. El Mesero recibe el aviso de que necesita al Encargado.
3. **Con preparación iniciada y autorizada por el Encargado:** el Pedido se cancela y registra motivo y autorizador. El handler pide al Inventario

registrar la merma de lo ya preparado (RN-11), guarda Pedido e Inventario en la misma unidad de trabajo, la confirma y publica `PedidoCancelado` y `MermaRegistrada`.

La regla de qué rol puede solicitar o autorizar vive en el control de acceso (capa de aplicación), y la regla de cuándo se exige autorización vive en el Pedido (dominio).



Código del diagrama de secuencia de cancelación en plantuml



Architecture Decision Records

Un ADR deja por escrito una decisión de arquitectura: por qué hacía falta, qué opciones se compararon, qué se eligió y qué se acepta a cambio. Redactamos tres, cada uno con su estado, contexto, alternativas, decisión, consecuencias y trazabilidad hacia requerimientos, reglas, historias y casos de uso.

Resumen de los ADR:

- **ADR-01, patrón de lógica de negocio:** Domain Model para Pedido, Promoción, Inventario, Entrega y Pago, y Table Module solo para reportes de lectura. Se acepta más estructura inicial a cambio de que cada regla viva en un solo lugar.
- **ADR-02, Repository y Unit of Work:** un repositorio por aggregate, con interfaz en el dominio, y una unidad de trabajo por solicitud aunque toque varios aggregates. Se acepta una transacción más amplia y la ausencia de control de concurrencia.
- **ADR-03, CQRS básico:** nueve commands y cuatro queries con DTO, sobre una sola base de datos. Se acepta tener más clases a cambio de que las pantallas no dependan de los aggregates.

ADR-01, patrón de lógica de negocio

Campo	Contenido
Título	Domain Model como patrón principal; Table Module solo para consultas agregadas
Estado	Aceptado
Contexto	Sazón Urbano vende por mesa, teléfono/WhatsApp y domicilio, con 80 a 150 pedidos diarios y más de 150 ingredientes. Las reglas más delicadas

	dependen del estado de las cosas: cancelar según si cocina ya empezó, bloquear una promoción sin stock, dejar una transferencia pendiente hasta que alguien la verifique. Esas reglas deben cumplirse igual en cualquier canal, y el gerente quiere cambiar promociones y puntos de lealtad sin reprogramar el sistema.
Alternativas	Transaction Script: rápido de escribir, pero cada flujo repetiría las validaciones de estado y de stock. Table Module: cómodo para reportes, pero no tiene un lugar natural para el ciclo de vida del pedido ni del pago. Domain Model: exige más clases al inicio, pero deja cada regla junto al dato que protege.
Decisión	Domain Model para Pedido, Promoción, Inventario, Entrega y Pago. Table Module solo para lecturas agregadas como el cierre de caja y el reporte de ventas. Transaction Script no se usa en los cinco casos porque ninguno es una operación lineal sin reglas.
Consecuencias	Beneficios: un cambio de regla toca una sola clase y el comportamiento es igual en los tres canales. Costos: más clases y más estructura que con scripts. Riesgos: Pedido e Inventario concentran comportamiento (revisión SOLID, H2). Trabajo posterior: si los reportes crecen, moverlos a un modelo de lectura propio.
Trazabilidad	RF-01 a RF-05, RF-12 y RF-15 · RNF-05 · Historias 1, 5, 9, 10, 12 y 16 · CU-01 a CU-05 · RN-01 a RN-17

ADR-02, Repository y Unit of Work

Campo	Contenido
Título	Un repositorio por aggregate y una Unit of Work por solicitud
Estado	Aceptado
Contexto	Varias operaciones cambian más de un aggregate a la vez: cancelar un pedido (Pedido e Inventario), confirmar una transferencia (Pago y Cliente) y cerrar una entrega (Entrega, Pedido y Repartidor). El BRD pide modo offline con sincronización posterior (RNF-07) y el local tiene WiFi básico, así que la forma de guardar los datos puede cambiar con el tiempo.
Alternativas	SQL directo dentro de los servicios: acopla el negocio a la tecnología. Un repositorio genérico para todo: pierde el lenguaje del negocio. Un aggregate por transacción con el resto vía eventos: más puro, pero deja estados intermedios visibles. Repositorio por aggregate más Unit of Work por solicitud.
Decisión	Siete repositorios (trece métodos) con interfaz en el dominio e implementación en infraestructura, en memoria para el código base. Una

	Unit of Work por solicitud: se confirma al final o se deshace todo, y los eventos se publican solo después de confirmar. Sin protección contra concurrencia, para mantener el diseño simple.
Consecuencias	Beneficios: se puede cambiar de tecnología o habilitar el modo offline sin tocar el negocio, y cancelar nunca deja un pedido sin su ajuste de inventario. Costos: más interfaces y transacciones más amplias de lo que recomienda DDD estricto. Riesgos: sin control de concurrencia, dos usuarios pueden actuar sobre un estado desactualizado (por ejemplo, cancelar mientras cocina inicia); además, un Inventario único carga más de 150 ingredientes en cada operación. Trabajo posterior: agregar versionado del pedido y evaluar consistencia eventual si el negocio crece.
Trazabilidad	RNF-06 y RNF-07 · RN-04, RN-11 y RN-15 · Historias 5, 9, 12 y 16 · Hallazgos H2 y H5 de la revisión SOLID

ADR-03, CQRS básico

Campo	Contenido
Título	CQRS básico con handlers, DTO de lectura y una sola base de datos
Estado	Aceptado
Contexto	Hay operaciones que cambian el estado (registrar, cancelar, cobrar, asignar) y consultas con necesidades distintas: resumen del pedido, menú disponible, historial del cliente y cierre de caja. Cada rol ve información diferente (el mesero sus pedidos, el encargado inventario y caja, el gerente los reportes) y el BRD pide respuestas en menos de 2 segundos en operaciones críticas (RNF-01).
Alternativas	Un solo modelo para leer y escribir: expondría los aggregates a las pantallas. CQRS completo con bases separadas: complejidad que un restaurante único no justifica. CQRS básico: separación en el código sobre una sola base.
Decisión	Nueve commands y cuatro queries, cada uno con su handler. Las queries dependen de interfaces de lectura y devuelven DTO, nunca el aggregate. El cierre de caja se trata como lectura agregada. Una sola base de datos.
Consecuencias	Beneficios: responsabilidades claras, pantallas independientes del dominio y cumplimiento del principio I de SOLID. Costos: más clases, un handler por operación. Riesgos: duplicar conceptos entre el DTO y el aggregate. Trabajo posterior: separar el almacén de lectura si los reportes se vuelven pesados.
Trazabilidad	RF-02, RF-08, RF-10, RF-15 y RF-16 · RNF-01 · Historias 2, 11 y 13 · Pregunta P15 de la entrevista

Validación de la propuesta

Antes de entregar volvimos sobre cada caso de uso para comprobar que el diseño aguantara de punta a punta: quién recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se guardan los cambios y qué se devuelve. Después aplicamos las seis validaciones de la guía (trazabilidad, lenguaje, responsabilidad, consistencia, persistencia y transacción) y una séptima sobre el uso de la IA.

- Trazabilidad, responsabilidad, consistencia, persistencia y transacción: cumplidas.
- Lenguaje: no del todo. Encontramos tres desfases entre documentos que quedaron registrados como hallazgos (ver tabla).
- Al revisar las etapas entre sí corregimos cuatro inconsistencias: el número de métodos de repositorio (eran trece, no catorce), el momento en que se acreditan los puntos de lealtad, quién actualiza el inventario al cancelar y un método que faltaba en Pedido para validar el cobro.
- Quedan declarados como supuestos pendientes de validar los que aparecen al final de esta sección.

Recorrido de los casos de uso sobre el diseño

Caso de uso	¿Quién recibe la solicitud?	¿Dónde se valida cada regla?	¿Qué objetos cambian?	¿Cómo se persiste?	¿Qué se devuelve?
CU-01 Registrar pedido	PedidoApplicationService → RegistrarPedidoHandler	Pedido: stock (RN-01), teléfono y dirección en domicilio (RN-02), responsable y fecha (RN-05). El permiso por rol, en ControlDeAcceso	Pedido nuevo con sus líneas	PedidoRepository.guardar dentro de la Unit of Work; PedidoConfirmado se publica después de confirmar	Id, total y estado. Errores: producto agotado, datos de domicilio incompletos
CU-02 Aplicar promoción	PedidoApplicationService → AplicarPromocionHandler	Promoción (vía AplicadorDePromocion): vigencia, productos y stock (RN-09). Pedido: una sola promoción (RN-06)	Pedido (promoción y total)	PedidoRepository.guardar en una Unit of Work	Pedido con descuento. Errores: fuera de vigencia, stock insuficiente, ya tiene

					promoción
CU-03 Cancelar pedido	PedidoApplicationService → CancelarPedidoHandler	Pedido: cancelación libre o con autorización según preparación (RN-03, RN-04). Inventario: devolución o merma (RN-11)	Pedido e Inventario	Ambos repositorios en una sola Unit of Work	Cancelación con o sin merma. Error: requiere autorización
CU-04 Registrar pago	PagoApplicationService → RegistrarPagoHandler o ConfirmarTransferenciaHandler	Pedido: listo para cobro. Pago: cada método define su estado inicial y su validación (RN-15, RN-16). Cliente: política de puntos (RN-17)	Pago; al confirmar transferencia, Pago y Cliente	PagoRepository (y ClienteRepository) en la Unit of Work; PagoConfirmado tras confirmar	Pago con su estado y el cambio en efectivo. Errores: monto insuficiente, referencia obligatoria
CU-05 Asignar repartidor y registrar entrega	EntregaApplicationService → AsignarRepartidorHandler y los handlers de salida, confirmación y falla	Entrega y AsignadorDeEntregas: disponibilidad, zona y solo domicilio (RN-12, RN-14). Entrega: evidencia obligatoria (RN-13), motivo si falla	Entrega y Repartidor; al cerrar, también Pedido	Los tres repositorios en una Unit of Work	Entrega asignada, en ruta, confirmada o fallida, con sus errores

Tabla de validación

Validación	**Pregunta**	**Resultado del equipo**

Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	**Sí.** Cada aggregate, servicio y handler se liga a un requerimiento o historia. Los componentes de soporte (PublicadorDeEventos, Notificador, Dinero) se justifican por los eventos, los avisos y los importes
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	**No.** Hay tres desfases: el BRD agrupa Mesero y Caja, pero las historias los distinguen; el glosario pide foto de merma obligatoria y la entrevista (P8) la deja opcional; la Historia 8 actualiza inventario al entregar y la Historia 5 al preparar. Quedan como hallazgos de la Etapa 1
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación?	**Sí.** Las reglas viven en Pedido, Promoción, Inventario, Entrega, Pago y Cliente. Las excepciones están justificadas: permisos por rol (capa de aplicación) y lecturas agregadas (Table Module)
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	**Sí.** Los participantes de las dos secuencias existen en el diagrama de clases y no hay clases sueltas. Se corrigieron cuatro inconsistencias entre etapas
Persistencia	¿El dominio evita depender directamente de SQL, ORM o infraestructura?	**Sí.** Dominio y aplicación solo conocen interfaces; las implementaciones quedan en infraestructura
Transacción	¿El límite de la operación y su manejo de error están definidos?	**Sí.** Una Unit of Work por solicitud: se confirma al final o se deshace todo, y los eventos salen después de confirmar. Los errores se dividen en de negocio y técnicos
Uso de IA	¿Se registró cómo se usó la IA y qué decidió el equipo?	**Sí.** Cada propuesta se contrastó con el BRD y con la guía. Los errores detectados, los supuestos y las decisiones están en la bitácora

Supuestos pendientes de validar

- Las reglas de negocio están numeradas por nosotros (RN-01 a RN-17) porque el BRD no las numera.
- Existe un solo inventario por restaurante.
- El inventario se descuenta al pasar el pedido a “En preparación”; ese paso queda fuera de los cinco casos de uso y se maneja por evento.
- Los puntos de lealtad se acreditan al recibir PagoConfirmado.
- El repartidor queda disponible al cerrar la entrega, sea confirmada o fallida.

- En domicilio con pago en efectivo, el cobro contra entrega (P11) se registra desde la Caja cuando el repartidor entrega el dinero.
- Los estados del pedido se derivan del flujo del BPMN y no del BRD.

Bitácora de IA

Usamos Claude (Anthropic) como apoyo en cada etapa: le entregamos el BRD, las historias y el diagrama de dominio, pedimos una propuesta con sus supuestos, la comparamos con nuestro material y con la guía, corregimos lo que no cuadraba y registramos el resultado. La IA propuso; el equipo decidió.

Tabla de iteraciones de la bitácora

Objetivo	Prompt (resumen)	Respuesta útil	Error o supuesto	Decisión del equipo
Planear el reporte	Pedir una guía de la estructura del reporte para armar el propio	Guía de las nueve etapas, adaptada al BRD de Sazón Urbano	El BRD no numera las reglas; se propusieron RN provisionales	Definir y numerar las reglas en la Etapa 1
Mapa de dominio	Desarrollar las fichas de aggregates sobre nuestro diagrama de draw.io	Siete fichas, value objects, tres domain services, eventos y observaciones al diagrama	Historia 8 y Historia 5 se contradicen sobre cuándo se actualiza el inventario	Conservar los siete aggregates del diagrama y registrar la contradicción como hallazgo
Service Layer	Pasar el mapa a servicios de aplicación	Tres servicios con un método por caso de uso; los multipaso enrutan solicitudes	Supuesto: el servicio lee el stock con una interfaz de solo lectura y se lo pasa al Pedido	Aprobar los tres servicios
Repository y Unit of Work	Definir repositorios y transacciones	Siete repositorios y tabla de qué se guarda junto	Error: el texto decía catorce métodos y eran trece. Contradicción entre puntos por evento y por misma transacción	Corregir a trece y resolver lo de los puntos en el ADR-02
Commands y Queries	Convertir los servicios en commands y queries	Nueve commands, cuatro queries y cuatro DTO	Supuesto: aparecen tres interfaces de lectura nuevas que hay que dibujar en el diagrama	Aprobar las tablas y agregar las interfaces al diagrama

Revisión con SOLID	Revisar el diseño con los cinco principios	Hallazgos por principio y tres correcciones (pagos, lealtad, contrato de inventario)	La primera versión se parecía demasiado en estructura a otro material de referencia	Pedir una versión con otra estructura y redacción propia
Diagramas UML	Generar el código PlantUML de clases y de dos secuencias	Diagrama de clases por capas y secuencias de pago y de cancelación	Error: Etapa 4 decía que el inventario se actualiza por evento y el diagrama lo hace el handler	Dejar el handler dentro de la Unit of Work y ajustar la Etapa 4
ADR y validación	Redactar los ADR y la validación	Tres ADR, recorrido de casos de uso, tabla de validación y supuestos	Supuesto: el cobro contra entrega se registra desde la Caja	Aprobar y dejar el supuesto como pendiente

Errores cometidos por la IA

#	Error	Cómo se detectó	Corrección
1	Dijo que los repositorios sumaban catorce métodos cuando la tabla suma trece	Al sumar los métodos de la tabla	Se corrigió el texto a trece
2	Escribió que los puntos de lealtad se acreditan por evento en la Etapa 4, pero los guardó en la misma transacción en la Etapa 5	Revisión cruzada de etapas	Se declaró el supuesto y se decide en el ADR-02
3	En la Etapa 4 dijo que el inventario se actualiza al recibir el evento de cancelación, pero el diagrama de secuencia lo hace el handler	Al armar las secuencias	Se unificó: lo hace el handler en la misma Unit of Work
4	El Pedido no tenía el método que valida que esté listo para cobro y la secuencia de pago lo usaba	Al armar la secuencia de pago	Se agregó el método al diagrama de clases
5	La primera versión de la revisión SOLID se parecía demasiado en estructura a otro material	El equipo lo notó al compararla	Se reescribió con otra estructura y redacción propia

6	Numeró las reglas de negocio (RN) por su cuenta, aunque el BRD no las numera	Al contrastar con el BRD	Se declararon como provisionales y se numeran en la Etapa 1

Tabla de decisiones del equipo

Decisión	**Efecto**
Mantener siete aggregates (incluyendo Pago, Cliente e Inventario)	El diseño cubre cobros, lealtad e inventario además del pedido y la entrega
Un método de servicio por caso de uso y tres servicios	Servicios delgados que solo enrutan
Usar la cancelación de pedido como segunda secuencia	Muestra una regla con excepción y alternativas
Reescribir la revisión SOLID con estructura propia	Redacción y orden distintos a los del material de referencia
Aceptar la ausencia de control de concurrencia	Diseño más simple, con el riesgo documentado en el ADR-02
Usar PlantUML para todos los diagramas	Cumple con lo que pide la actividad

Repositorio

https://github.com/Landon919/Equipo_3.git

Conclusiones

Karla Judith Baez Villanueva

El desarrollo del modelo de dominio me permitió comprender la importancia de definir con precisión dónde reside cada regla de negocio antes de programar. Encapsular la lógica en entidades como Pedido e Inventario garantiza que el sistema mantenga una consistencia estricta y evite estados inválidos en la operación del restaurante, logrando una arquitectura limpia, modular y fácil de mantener.

Alexa Daiana Amaya Torres:

La elaboración del modelo de dominio me ayudó a entender la importancia de establecer correctamente las reglas de negocio antes de comenzar con la programación. Organizar la lógica en entidades como Pedido e Inventario permite controlar mejor la información, prevenir errores y asegurar el buen funcionamiento del sistema del restaurante. Además, facilita que el programa sea más ordenado, flexible y sencillo de actualizar en el futuro.

Landon Alexander Bautista Hernandez

Este trabajo me permitió ver que diseñar la arquitectura de un sistema va mucho más allá de escribir código. Primero hay que decidir cómo se va a organizar la lógica de negocio. Al comparar patrones como Transaction Script y Domain Model, entendí que no existe uno "mejor" en general. La elección depende de qué tan complejas sean las reglas del caso. Para un sistema con pedidos, pagos, promociones y entregas, Domain Model resultó más adecuado porque concentra las reglas en el dominio y evita que se dispersen.

Reporte